Brandon Gilberto Imanol Retana Fuentes - AL07175444
Máster en Inteligencia Artificial
Gestión de Proyectos de Inteligencia Artificial
Módulo 2

Fase 2. Ejecución - “ClaimCheck AI”

“Manual de despliegue en la nube: contenedor Docker y plataforma PaaS”

Resumen

- **Qué se despliega:** la API REST de ClaimCheck IA (FastAPI), su frontend web, el modelo `claimcheck_router_v1` y un motor de 14 reglas nutrimentales, todo en una sola imagen Docker de tres etapas.
- **Estrategia:** contenedor + PaaS en nube pública. Destino principal: Render (plan Free, construye el Dockerfile desde GitHub). Alternativa documentada: Google Cloud Run.
- **Requisitos medidos:** ~99 MB de RAM, arranque de 3.2 s con 0.5 CPU y de 19.2 s con 0.1 CPU, p95 de 95 ms con los recursos del plan Free.
- **Calidad integrada:** la imagen solo se construye si pasan las 84 pruebas; GitHub Actions valida el contenedor en cada push y Render despliega solo con los checks en verde.

1. Propósito y alcance

Este manual describe, paso a paso, cómo llevar ClaimCheck IA desde el repositorio de GitHub hasta un servicio público con HTTPS. Está escrito para que otra persona del equipo técnico pueda repetir el despliegue sin ayuda y obtener el mismo resultado: la misma imagen, el mismo modelo (verificado por su huella SHA-256) y las mismas respuestas.

Alcance: construcción y prueba de la imagen, ejecución local, despliegue en Render, alternativa en Google Cloud Run, verificación posterior, actualización, reversión, monitoreo y retiro del servicio. **Fuera de alcance:** autenticación de usuarios y manejo de etiquetas reales de clientes, que se abordan como requisitos antes de un piloto con datos confidenciales (sección 6).

El documento sigue el marco del ciclo de vida de ISO/IEC 23053:2022, que pide documentar el propósito, el diseño controlado, la verificación y validación, la supervisión operativa y el fin de vida del sistema (SG Systems, 2025). La sección 8 relaciona cada etapa con su evidencia en el repositorio.

2. Descripción de la solución

ClaimCheck IA preclasifica los claims de una etiqueta de alimentos para el organismo certificador ficticio CertiBajío, S.C. El modelo enruta cada claim al checklist que le corresponde (NUTRIMENTAL, SALUD, AUSENCIA o MARKETING) y el motor de reglas compara la tabla nutrimental contra los umbrales del Anexo del Reglamento (CE) 1924/2006. El principio de la Fase 1 se mantiene: **la IA propone y el revisor técnico decide**.

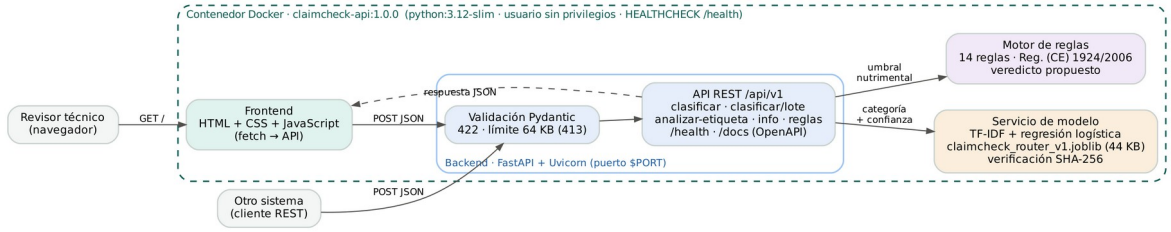


Figura 1. Arquitectura: frontend, API, modelo y reglas dentro de un solo contenedor.

Componente	Tecnología	Función en el despliegue
Frontend	HTML + CSS + JavaScript (`Fetch`)	Interfaz del revisor; la sirve la propia API en `/` (mismo origen, sin CORS)
Backend / API REST	FastAPI 0.162 + Uvicorn 0.54	Expone 6 endpoints y el contrato OpenAPI en `/docs`; valida toda entrada con Pydantic
Modelo	scikit-learn 1.9.1 (TF-IDF + regresión logística)	Artefacto de 44 KB; se verifica su SHA-256 antes de cargarlo
Motor de reglas	Python	14 reglas deterministas del Reg. (CE) 1924/2006
Contenedor	Docker, imagen base `python:3.12-slim`	Empaqueta todo; corre como usuario sin privilegios con `HEALTHCHECK`
CI / CD	GitHub Actions + Render	Prueba el contenedor en cada push y despliega si los checks pasan

Tabla 1. Componentes de la solución y su función en el despliegue.

Contrato de la API

Método	Ruta	Respuesta
GET	`/health`	Estado del servicio, versión del modelo e integridad (usado por Docker y por la PaaS)
GET	`/api/v1/info`	Metadatos: algoritmo, métricas de prueba, umbral τ, huella del artefacto
GET	`/api/v1/reglas`	Catálogo de las 14 reglas
POST	`/api/v1/clasificar`	Un claim → categoría, confianza, checklist, `requiere_confirmacion` y motivos
POST	`/api/v1/clasificar/lote`	Hasta 50 claims por llamada
POST	`/api/v1/analizar-etiqueta`	Hasta 30 claims + tabla nutrimental → enrutamiento, veredictos de regla y resumen

Tabla 2. Contrato de la API REST.

3. Requerimientos técnicos

3.1 Software y cuentas

Elemento	Versión o detalle	Para qué
Docker Desktop (Windows con WSL 2 o macOS) o Docker Engine (Linux)	Reciente, con Compose v2	Construir y probar la imagen en local
Git y cuenta de GitHub	Repositorio `BGIRF_Proyecto_Fase2_GPIA`	Fuente única del código; dispara la CI y el despliegue
Cuenta de Render	Plan Free para servicios web	Plataforma PaaS principal
Python	3.9 o superior, sin paquetes adicionales	Script de validación de punta a punta
Google Cloud CLI (opcional)	`gcloud` y un proyecto con facturación habilitada	Solo para la alternativa en Cloud Run

Tabla 3. Software y cuentas requeridos.

3.2 Recursos de cómputo

Los valores siguientes se midieron sobre el contenedor en ejecución con los límites indicados (sección 8 del documento de validación y pruebas). Caben con holgura en el plan gratuito de Render, que ofrece 0.1 CPU y 512 MB (Render, s.f.-d).

Recurso	Local (0.5 CPU · 512 MB)	Simulación plan Free (0.1 CPU · 512 MB)
Memoria en reposo	99 MB (19 % del límite)	99 MB (19 % del límite)
Tiempo hasta responder `/health`	3.2 s	19.2 s
Latencia de `/clasificar` (p50 / p95)	4.9 / 44.3 ms	13.1 / 95.3 ms
Rendimiento con concurrencia	128 solicitudes/s, 0 errores	22 solicitudes/s, 0 errores
Disco	Artefacto de 44 KB; imagen dominada por Python, NumPy y SciPy	Igual

Tabla 4. Recursos medidos sobre el contenedor en ejecución.

3.3 Dependencias fijadas

Las versiones están fijadas en `requirements.txt` porque el artefacto `.joblib` se serializó con ellas: cambiar scikit-learn, NumPy o SciPy puede impedir que el modelo cargue. Al arrancar, el servicio compara la versión de scikit-learn con la registrada en los metadatos y advierte si difieren.

```
fastapi==0.162.0    uvicorn==0.54.0    pydantic==2.14.2
scikit-learn==1.9.1  numpy==2.5.4      scipy==1.18.1     joblib==1.6.0
```

3.4 Variables de entorno

Variable	Por defecto	Descripción
`PORT`	8000	Puerto en que escucha Uvicorn. Render lo fija en 10000 y Cloud Run en 8080; no hay que cambiar nada
`WEB_CONCURRENCY`	1	Procesos de Uvicorn; con 512 MB se recomienda 1
`LOG_LEVEL`	INFO	Detalle de los logs (DEBUG, INFO, WARNING)
`UMBRAL_CONFIANZA`	vacío → 0.35	Umbral τ para marcar claims a confirmar; vacío usa el del entrenamiento
`ORIGENES_PERMITIDOS`	vacío	Orígenes CORS, solo si el frontend se aloja en otro dominio
`TAMANO_MAXIMO_BYTES`	65536	Tamaño máximo del cuerpo de una solicitud (respuesta 413 si se excede)

Tabla 5. Variables de entorno del contenedor.

3.5 Red y puertos

- El servicio escucha en `0.0.0.0:$PORT`, como exigen las plataformas: Render reenvía el tráfico al puerto de la variable `PORT`, cuyo valor por defecto es 10000 (Render, s.f.-c).
- HTTPS lo termina la plataforma; el contenedor solo atiende HTTP interno. Uvicorn arranca con `--proxy-headers` para registrar la IP real detrás del proxy.
- En local se publica con `-p 8000:8000`. El `HEALTHCHECK` consulta `http://127.0.0.1:$PORT/health` dentro del contenedor.

4. Estrategia de despliegue

4.1 Modelo de implementación: nube pública

Para esta fase se elige **nube pública**: el dataset es propio y ficticio, no hay datos personales ni etiquetas de clientes, y la prioridad es costo cero y escalabilidad sin administrar servidores. La nube pública es la opción indicada cuando no se maneja información confidencial (Martínez, 2024).

Para producción la decisión cambia: las etiquetas de un cliente antes de su lanzamiento son información confidencial. La recomendación es una **nube híbrida**: los documentos del cliente se quedan en un entorno privado o en una región con acuerdos de confidencialidad, y solo el texto de los claims se envía al servicio de clasificación, que no almacena nada. Como el contenedor es portable, el mismo Dockerfile corre en cualquiera de los dos entornos.

4.2 Modelo de servicio: contenedor sobre PaaS

Opción	Ejemplo	Ventajas	Desventajas para este proyecto	Decisión
IaaS	Máquina virtual (AWS EC2)	Control total del sistema operativo y la red	Hay que instalar Docker, parchar, abrir puertos y gestionar HTTPS	Descartada
PaaS de contenedores	Render · Google Cloud Run	Construye el Dockerfile, HTTPS y health checks incluidos, escala sin administrar servidores	Arranque en frío en planes gratuitos	**Elegida**
FaaS	AWS Lambda · Cloud Functions	Pago por invocación	El modelo y NumPy/SciPy exceden el paquete típico; arranque en frío en cada escala	Descartada
SaaS	API de terceros	Sin operación	No existe un servicio que haga este enrutamiento con nuestros checklists	No aplica

Tabla 6. Comparación de modelos de servicio en la nube.

Pasar de IaaS a PaaS reduce la parte de la infraestructura que gestiona el equipo y traslada esa responsabilidad al proveedor (China y Goodwin, s.f.). Para un equipo pequeño con un servicio sin estado y ligero, eso pesa más que el control fino de una máquina virtual.

4.3 Plataforma: Render (principal) y Google Cloud Run (alternativa)

Criterio	Render · plan Free	Google Cloud Run
Cómo construye	Desde el repositorio de GitHub con el Dockerfile (BuildKit) (Render, s.f.-b)	`gcloud run deploy --source .` usa Cloud Build y el Dockerfile
Recursos	0.1 CPU · 512 MB (Render, s.f.-d)	Configurables (aquí 1 vCPU · 512 MiB)
Costo	$0; 750 horas de instancia por mes y espacio de trabajo (Render, s.f.-a)	Capa gratuita mensual: 2 millones de solicitudes, 180 000 vCPU-s y 360 000 GiB-s (Google Cloud, s.f.)
Inactividad	Se suspende tras 15 min sin tráfico; reactivarlo toma cerca de 1 min (Render, s.f.-a)	Escala a cero; arranque en frío de segundos
Requisitos	Cuenta y conexión con GitHub	Proyecto con cuenta de facturación
Por qué	Plan gratuito, conexión directa con GitHub y archivo `render.yaml` versionado	Mejor opción si se necesita más CPU o una región concreta

Tabla 7. Render frente a Google Cloud Run.

Hugging Face Spaces, que se usó para la demo de la Actividad 4, ya no es una opción gratuita para este caso: crear un Space que ejecute un contenedor Docker requiere un plan de pago (Hugging Face, s.f.).

4.4 Flujo de integración y despliegue continuos



Figura 2. Flujo CI/CD: cada push se prueba en GitHub Actions y Render despliega solo con los checks en verde.

- **Compuerta 1 · build:** la etapa final del Dockerfile copia un archivo de la etapa `test`, así que Docker no puede producir la imagen si alguna de las 84 pruebas falla. Aplica en local, en GitHub Actions y en Render.
- **Compuerta 2 · CI:** `.github/workflows/ci.yml` construye la imagen, la levanta con 512 MB, espera el estado `healthy` y corre las 50 pruebas de `scripts/validar_contenedor.py`.
- **Compuerta 3 · CD:** `render.yaml` usa `autoDeployTrigger: checksPass`, que despliega solo cuando los checks del commit pasan (Render, s.f.-d).

4.5 Versionado y reversión

- **Código:** cada despliegue corresponde a un commit; Render muestra el commit desplegado.
- **Imagen:** etiqueta semántica (`claimcheck-api:1.0.0`) y metadatos OCI (`LABEL`) con versión, autor y repositorio.
- **Modelo:** `modelos/claimcheck_router_v1.json` registra versión, fecha, hiperparámetros, métricas, versiones de librerías y SHA-256 del artefacto y del dataset. Un modelo nuevo se publica como `v2` con su propio archivo de metadatos; si falla, se regresa al commit anterior.

5. Procedimiento paso a paso

Paso 1 · Publicar el repositorio en GitHub

1. Crea el repositorio `BGIRF_Proyecto_Fase2_GPIA` en GitHub (público, para que la evaluación pueda consultarlo).
2. Sube el contenido completo del proyecto. Verifica que se subieron los archivos que empiezan con punto: `.dockerignore`, `.gitignore`, `.env.example` y la carpeta `.github/workflows/`.
3. En la pestaña **Actions**, confirma que el flujo *CI · pruebas y contenedor* termina en verde. Si falla, el detalle de cada paso indica la prueba afectada.

Paso 2 · Construir y validar en local

Antes de tocar la nube se comprueba que la imagen funciona en el equipo propio. El detalle, con PowerShell y solución de problemas, está en `INSTRUCCIONES_DOCKER.md`.

```
docker build -t claimcheck-api:1.0.0 .                  # incluye las 84 pruebas
docker run -d --name claimcheck -p 8000:8000 claimcheck-api:1.0.0
docker ps                                               # STATUS: Up … (healthy)
python scripts/validar_contenedor.py                    # 50/50 pruebas aprobadas
```

Paso 3 · Simular las condiciones de la nube

Se repite la ejecución con los recursos del plan Free y con el puerto que asigna Render. Así se detectan en local los problemas de memoria, de CPU o de enlace de puerto.

```
docker run -d --name claimcheck-free -p 10000:10000 -e PORT=10000 \
  --memory 512m --cpus 0.1 claimcheck-api:1.0.0
```

```
python scripts/validar_contenedor.py --url http://localhost:10000 --etiqueta
simulacion_render_free
```

Paso 4 · Crear el servicio en Render

Opción A (recomendada): Blueprint con render.yaml

1. Entra a dashboard.render.com con tu cuenta de GitHub y autoriza el acceso al repositorio.
2. Elige **New → Blueprint** y selecciona `BGIRF_Proyecto_Fase2_GPIA`. Render detecta `render.yaml`.
3. Revisa el servicio propuesto (`claimcheck-api`, Docker, plan Free, health check `/health`) y aplica el Blueprint.

```
services:
  - type: web
    name: claimcheck-api
    runtime: docker
    plan: free                    # 0.1 CPU · 512 MB; se suspende tras 15 min sin tráfico
    region: ohio
    dockerfilePath: ./Dockerfile
    dockerContext: .
    healthCheckPath: /health
    autoDeployTrigger: checksPass  # despliega solo si pasan las pruebas de GitHub Actions
    envVars:
      - key: LOG_LEVEL
        value: INFO
      - key: WEB_CONCURRENCY
        value: "1"
```

Opción B: servicio web manual

1. **New → Web Service** y conecta el repositorio.
2. **Language:** Docker (Render construye la imagen con el Dockerfile de la raíz). **Instance Type:** Free.
3. En **Advanced**, escribe `/health` como **Health Check Path** y agrega `LOG_LEVEL=INFO`. No definas `PORT`: Render lo asigna.
4. Confirma la creación del servicio; Render inicia el primer despliegue.

Paso 5 · Seguir la construcción y el arranque

En los logs del despliegue deben aparecer, en este orden, las señales de la tabla. Cuando Render marca el servicio como *Live*, la URL pública queda activa (por ejemplo, `https://claimcheck-api.onrender.com`; el subdominio depende de que el nombre esté disponible).

Señal en el log	Significado
`84 passed`	La etapa `test` pasó; la imagen puede construirse
`Modelo 1.0.0 cargado (4 clases, τ=0.35, SHA-256 verificado)`	El artefacto es el registrado y se cargó
`Uvicorn running on http://0.0.0.0:10000`	El servicio escucha en el puerto que asignó Render
`GET /health … 200`	El health check de la plataforma responde

Tabla 8. Señales esperadas en el log del despliegue.

Paso 6 · Verificación posterior al despliegue

Se corre la misma batería de 50 pruebas contra la URL pública. La primera solicitud después de 15 minutos sin tráfico puede tardar cerca de un minuto mientras el servicio despierta; el script espera hasta 30 s por solicitud, así que conviene abrir antes la URL en el navegador.

```
python scripts/validar_contenedor.py --url https://claimcheck-api.onrender.com --etiqueta render
```

- Abre la URL en el navegador, presiona **Cargar ejemplo** y **Analizar etiqueta**.
- Abre `/docs` y prueba `POST /api/v1/clasificar` con *Try it out*.
- Guarda `resultados/validacion_render.json` y una captura de la pantalla de resultados como evidencia del despliegue.

Paso 7 · Actualizar, revertir y retirar

Operación	Procedimiento
Actualizar	Haz push a GitHub. Actions prueba el contenedor y, con los checks en verde, Render construye y despliega la nueva versión sin intervención.
Cambiar el modelo	Reentrena con `python -m src.entrenar_modelo`, revisa las métricas en el JSON de metadatos, corre las pruebas y sube el nuevo artefacto junto con su JSON. Si las pruebas de regresión fallan, la imagen no se construye.
Revertir	En el historial de despliegues de Render elige el despliegue anterior y usa la reversión (rollback); o revierte el commit en GitHub, lo que dispara un despliegue nuevo con la versión previa.
Retirar (fin de vida)	Suspende o elimina el servicio en Render, revoca el acceso de Render al repositorio y archiva el repositorio. Los metadatos del modelo y este manual se conservan como registro (ISO/IEC 23053).

Tabla 9. Operaciones del ciclo de vida del servicio.

Alternativa · Google Cloud Run

Requiere un proyecto de Google Cloud con facturación habilitada y la CLI `gcloud`. Cloud Build construye el mismo Dockerfile (incluida la etapa de pruebas) y Cloud Run inyecta `PORT=8080`.

```
gcloud auth login
gcloud config set project <ID_DEL_PROYECTO>
gcloud services enable run.googleapis.com cloudbuild.googleapis.com
artifactregistry.googleapis.com
gcloud run deploy claimcheck-api --source . --region us-central1 \
  --allow-unauthenticated --memory 512Mi --cpu 1 --max-instances 2
gcloud run services describe claimcheck-api --region us-central1 --format "value(status.url)"
```

`--max-instances 2` acota el costo ante un pico de tráfico. Para retirar el servicio: `gcloud run services delete claimcheck-api --region us-central1`.

6. Operación, seguridad y costos

6.1 Monitoreo

- **Salud:** `HEALTHCHECK` de Docker cada 30 s y health check de la plataforma sobre `/health`, que además informa si el modelo pasó la verificación de integridad.
- **Logs:** cada solicitud a `/api/` registra método, ruta, código, milisegundos e identificador `X-Request-ID`, que también se devuelve al cliente para rastrear un caso concreto.
- **Métricas a vigilar:** latencia p95 (umbral 300 ms), proporción de claims marcados para confirmación (17.8 % en la validación) y tasa de respuestas 4xx/5xx. Un aumento sostenido de claims marcados indica que los claims reales se alejan del dataset de entrenamiento.

6.2 Seguridad aplicada

Control	Implementación	Evidencia
Mínimo privilegio	Usuario `claimcheck` (uid 10001); código y modelo propiedad de root, solo lectura	Pruebas C-01 a C-03
Integridad del modelo	SHA-256 verificado antes de `joblib.load`; si no coincide, el servicio no arranca	Prueba C-04 y `test_artefacto_alterado_no_se_carga`
Validación de entradas	Pydantic: longitudes, tipos, rangos, campos no permitidos, masa físicamente imposible	Casos extremos E-01 a E-18
Límite de carga	Cuerpo máximo de 64 KB (413) y lotes de 50 claims	E-12 a E-14
Sin secretos en la imagen	Configuración por variables de entorno; `.env` excluido por `.gitignore` y `.dockerignore`	Revisión del Dockerfile
Cifrado en tránsito	HTTPS provisto por la plataforma	URL pública `https://`
Contenido no ejecutable	El frontend inserta texto con `textContent`, nunca como HTML	E-25 (inyección HTML/JS)

Tabla 10. Controles de seguridad aplicados.

Pendiente antes de un piloto con datos reales: autenticación (clave de API o SSO), límite de tasa por cliente, registro de auditoría de las decisiones del revisor y acuerdo de tratamiento de datos con el proveedor de nube.

6.3 Costos

El despliegue de esta fase cuesta $0 en el plan Free de Render. La instancia de pago más pequeña, sin suspensión por inactividad, cuesta $7 al mes con 0.5 CPU y 512 MB (temps.sh, 2026). Con la latencia medida, una sola instancia atiende con holgura las ~86 400 clasificaciones anuales del caso de negocio de la Fase 1.

7. Solución de problemas en la nube

Síntoma	Causa probable	Acción
El build falla en `RUN python -m pytest`	Una prueba no pasa	Revisa el nombre de la prueba en el log; corrige en local con `pytest` antes de volver a subir
Render reporta que no detecta un puerto abierto	El proceso no escucha en `0.0.0.0:$PORT`	No sobrescribas el `CMD`; verifica que no se fijó `PORT` con otro valor
El health check falla y el despliegue se revierte	El modelo no cargó (versión de librerías o archivo alterado)	Busca `ErrorIntegridad` o la advertencia de versión de scikit-learn en los logs
La primera solicitud tarda ~1 min	El servicio Free estaba suspendido	Comportamiento esperado; para evitarlo, usar una instancia de pago
Error de memoria (OOM)	Más procesos de los que caben en 512 MB	Mantén `WEB_CONCURRENCY=1`
El frontend en otro dominio no puede llamar a la API	CORS cerrado por diseño	Agrega el dominio en `ORIGENES_PERMITIDOS`

Tabla 11. Solución de problemas en la nube.

8. Trazabilidad con ISO/IEC 23053

Etapa del ciclo de vida	Evidencia en el repositorio
Propósito y requisitos	`README.md`; caso de negocio de la Fase 1; umbrales de negocio en los metadatos del modelo
Datos	`data/claims_etiquetado.csv` y su ficha `data/README.md`; SHA-256 del dataset en los metadatos

Etapa del ciclo de vida	Evidencia en el repositorio
Diseño y entrenamiento	`src/entrenar_modelo.py` (fuera de línea, semilla 42); hiperparámetros y validación cruzada en `modelos/claimcheck_router_v1.json`
Verificación y validación	`tests/` (84 pruebas), `scripts/validar_contenedor.py` (50 pruebas), `resultados/` y el documento de validación
Despliegue	`Dockerfile`, `docker-compose.yml`, `render.yaml`, `.github/workflows/ci.yml` y este manual
Operación y monitoreo	`/health`, `HEALTHCHECK`, logs con `X-Request-ID`, métricas de la sección 6.1
Fin de vida	Procedimiento de retiro del paso 7

Tabla 12. Trazabilidad con las etapas de ISO/IEC 23053.

9. Uso de IA generativa en la documentación

La documentación técnica (este manual, el README y el documento de validación) se elaboró con apoyo de IA generativa, siguiendo el enfoque del Tema 16: un prompt que especifica el tipo de proyecto y su alineación con ISO/IEC 23053. Se usó el asistente Claude (Anthropic); los mismos prompts funcionan en Microsoft Copilot. La IA estructuró y redactó; **todas las cifras provienen de los archivos de resultados del repositorio** y se verificaron contra ellos, y los comandos se ejecutaron antes de documentarse.

Uso	Prompt documentado
Estructura del manual	«Actúa como ingeniero MLOps. Redacta un manual de despliegue en la nube para una API FastAPI con un modelo scikit-learn en un contenedor Docker, alineado con el ciclo de vida de ISO/IEC 23053: propósito, requisitos técnicos, estrategia (modelo de nube y de servicio), procedimiento paso a paso, verificación, operación, reversión y retiro. Audiencia: equipo técnico de un organismo certificador.»
Estrategia de despliegue	«Compara IaaS, PaaS de contenedores, FaaS y SaaS para un servicio de inferencia sin estado de 100 MB de RAM, con plan gratuito como restricción. Entrega una tabla con ventajas, desventajas y decisión justificada.»
Runbook de fallas	«Con base en este Dockerfile y en estos logs, genera una tabla de solución de problemas para Render: síntoma, causa probable y acción.»
Revisión de consistencia	«Revisa que cada cifra del documento coincida con estos archivos JSON de resultados y señala cualquier diferencia.»

Tabla 13. Prompts documentados para la elaboración del manual.

10. Referencias

- China, C., y Goodwin, M. (s.f.). *IaaS, PaaS, SaaS: ¿cuál es la diferencia?* IBM. https://www.ibm.com/mx-es/think/topics/iaas-paas-saas
- Comisión Europea. (2006). Reglamento (CE) n.º 1924/2006 relativo a las declaraciones nutricionales y de propiedades saludables en los alimentos. *Diario Oficial de la Unión Europea*, L 404.
- Docker Inc. (s.f.). *Multi-stage builds*. https://docs.docker.com/build/building/multi-stage/
- Google Cloud. (s.f.). *Cloud Run pricing*. https://cloud.google.com/run/pricing
- Hugging Face. (s.f.). *Spaces overview*. https://huggingface.co/docs/hub/spaces-overview
- International Organization for Standardization. (2022). *ISO/IEC 23053:2022 Framework for artificial intelligence (AI) systems using machine learning (ML)*. ISO.
- Martínez, M. (2024). *¿Qué es la nube pública, privada e híbrida? ¿Cuáles son sus ventajas y desventajas?* NIC México.
- Render. (s.f.-a). *Deploy for free*. https://render.com/docs/free
- Render. (s.f.-b). *Docker on Render*. https://render.com/docs/docker
- Render. (s.f.-c). *Web services*. https://render.com/docs/web-services
- Render. (s.f.-d). *Blueprint YAML reference*. https://render.com/docs/blueprint-spec
- SG Systems. (2025). *ISO/IEC 23053 — Marco del ciclo de vida del sistema de IA*. https://sgsystemsglobal.com/es/glosario/
- temps.sh. (2026). *Render pricing 2026: Free tier sleep, plans, compute*. https://temps.sh/blog/render-pricing-2026